

Relatório - Compressão de Arquivos-Texto com Algoritmo de Huffman

Aluno(s): [PREENCHER]

Curso: Ciência da Computação

Disciplina: Projeto e Análise de Algoritmos

Professor: [PREENCHER]

Data de entrega: [PREENCHER]

1. Objetivo

O objetivo deste trabalho é implementar um programa em Java para compactação e descompactação de arquivos-texto utilizando o algoritmo de Huffman. O sistema permite realizar a codificação por caractere e por palavra, conforme solicitado no enunciado do trabalho.

O programa apresenta um menu com as seguintes opções:

1. Compactar arquivo usando codificação por caractere;
2. Descompactar arquivo usando codificação por caractere;
3. Compactar arquivo usando codificação por palavra;
4. Descompactar arquivo usando codificação por palavra.

Em todas as opções, o usuário informa o arquivo de entrada e o arquivo de saída.

2. Visão geral do algoritmo de Huffman

O algoritmo de Huffman é um método de compressão sem perda. Isso significa que, após a compactação e a descompactação, o arquivo reconstruído deve ser idêntico ao arquivo original.

A ideia principal é atribuir códigos menores aos símbolos que aparecem com maior frequência e códigos maiores aos símbolos menos frequentes. Dessa forma, o texto passa a ser representado com uma quantidade menor de bits.

O processo geral utilizado no programa é:

1. Ler os símbolos do arquivo;
2. Contar a frequência de cada símbolo;
3. Construir a árvore de Huffman com base nas frequências;
4. Gerar o dicionário de códigos binários;
5. Substituir cada símbolo pelo respectivo código Huffman;
6. Gravar os bits compactados no arquivo de saída;
7. Gravar também o dicionário no arquivo compactado, permitindo a descompactação posterior.

3. Codificação por caractere

Na codificação por caractere, cada caractere do arquivo é tratado como um símbolo individual. Isso inclui letras, acentos, espaços, pontuações e quebras de linha.

Exemplo:

pão

Símbolos considerados:

p
ã
o

A contagem de frequência é feita para cada caractere. Depois disso, a árvore de Huffman é construída e cada caractere recebe um código binário de tamanho variável.

4. Codificação por palavra

Na codificação por palavra, o programa separa o texto em tokens. Um token pode ser uma sequência de letras ou uma sequência de não letras.

Exemplo:

Olá, mundo!

Tokens considerados:

"Olá"
", "
"mundo"
"!"

Essa decisão foi adotada para preservar espaços, pontuação e quebras de linha, permitindo que a descompactação reconstrua o arquivo exatamente igual ao original.

5. Estruturas de dados utilizadas

As principais estruturas de dados utilizadas foram:

- `HashMap<String, Long>`: armazena a frequência de cada símbolo;
- `PriorityQueue<HuffmanNode>`: seleciona automaticamente os dois nós de menor frequência durante a construção da árvore;

- **HuffmanNode**: representa cada nó da árvore de Huffman;
 - **HashMap<String, String>**: representa o dicionário Huffman, associando cada símbolo ao seu código binário;
 - **DecodeNode**: representa a árvore de decodificação usada na descompactação;
 - **BitWriter**: agrupa os bits gerados em bytes reais antes de gravar no arquivo compactado;
 - **CompressedFileHeader**: cabeçalho salvo no arquivo compactado, contendo o modo de compressão, o dicionário Huffman e a quantidade de bits válidos.
-

6. Formato do arquivo compactado

O arquivo compactado gerado pelo programa contém duas partes principais:

1. Cabeçalho serializado em Java, contendo:
 - modo de compressão usado: caractere ou palavra;
 - dicionário Huffman;
 - quantidade de bits válidos na área compactada.
2. Sequência de bytes compactados, formada pelos códigos Huffman agrupados em bytes.

O dicionário é salvo no próprio arquivo compactado. Assim, a descompactação não depende de reconstruir a árvore a partir das frequências, evitando problemas quando símbolos possuem a mesma frequência.

7. Processo de compactação

A compactação segue as seguintes etapas:

1. O programa recebe o arquivo de entrada e o arquivo de saída;
 2. O arquivo é lido em UTF-8;
 3. O programa divide o conteúdo em símbolos, dependendo do modo escolhido;
 4. As frequências são contadas;
 5. A árvore de Huffman é construída com uma fila de prioridade;
 6. O dicionário é gerado percorrendo a árvore;
 7. O arquivo é lido novamente e cada símbolo é substituído pelo código correspondente;
 8. Os bits são agrupados de 8 em 8 para formar bytes;
 9. O cabeçalho e os bytes compactados são gravados no arquivo final;
 10. O programa exibe tamanho original, tamanho compactado, taxa de compressão, redução obtida e tempo de execução.
-

8. Processo de descompactação

A descompactação segue as seguintes etapas:

1. O programa lê o arquivo compactado;
 2. O cabeçalho é recuperado;
 3. O dicionário Huffman é utilizado para montar uma árvore de decodificação;
 4. Os bytes compactados são lidos bit a bit;
 5. A cada código completo encontrado, o símbolo original é escrito no arquivo de saída;
 6. O processo continua até atingir a quantidade de bits válidos registrada no cabeçalho;
 7. O arquivo reconstruído é salvo em UTF-8.
-

9. Mensuração do tempo de execução

O tempo de execução foi medido diretamente no programa utilizando `System.nanoTime()`.

A medição é iniciada imediatamente antes da operação de compactação ou descompactação e finalizada logo após a conclusão da operação. O resultado é convertido para milissegundos.

Fórmula usada no código:

```
tempo_ms = (tempo_final_ns - tempo_inicial_ns) / 1.000.000
```

Ambiente utilizado nos testes:

- Processador: [PREENCHER]
 - Memória RAM: [PREENCHER]
 - Sistema operacional: [PREENCHER]
 - Versão do Java: [PREENCHER]
 - Comando de execução usado: [PREENCHER]
-

10. Taxa de compressão

A taxa de compressão foi calculada dividindo o tamanho do arquivo compactado pelo tamanho do arquivo original.

```
taxa = tamanho_compactado / tamanho_original
```

A redução percentual foi calculada por:

```
redução = 1 - taxa
```

Exemplo:

Arquivo original: 100 MB
Arquivo compactado: 40 MB
Taxa: $40 / 100 = 0,40 = 40\%$
Redução: $1 - 0,40 = 0,60 = 60\%$

11. Resultados obtidos

11.1 Arquivo de aproximadamente 10 MB

Codificação por caractere

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Codificação por palavra

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Observações sobre o teste de 10 MB: [PREENCHER]

11.2 Arquivo de aproximadamente 50 MB

Codificação por caractere

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Codificação por palavra

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Observações sobre o teste de 50 MB: [PREENCHER]

11.3 Arquivo de aproximadamente 200 MB

Codificação por caractere

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Codificação por palavra

- Tamanho original: [PREENCHER]
- Tamanho compactado: [PREENCHER]
- Taxa compactado/original: [PREENCHER]
- Redução obtida: [PREENCHER]
- Tempo de compactação: [PREENCHER]
- Tempo de descompactação: [PREENCHER]

Observações sobre o teste de 200 MB: [PREENCHER]

12. Análise empírica de desempenho

Com base nos resultados obtidos, observou-se que: [PREENCHER]

Comparação entre compactação por caractere e por palavra: [PREENCHER]

Comportamento conforme o tamanho do arquivo aumenta: [PREENCHER]

Diferença entre tempo de compactação e descompactação: [PREENCHER]

Possíveis causas para os resultados observados: [PREENCHER]

13. Validação da descompactação

Para validar se a descompactação reconstruiu corretamente os arquivos originais, foi utilizado o seguinte método:

- Linux/macOS: comando `cmp`;
- Windows: comando `fc.exe /b`.

Resultados da validação:

- 10 MB, caractere: [PREENCHER]
- 10 MB, palavra: [PREENCHER]
- 50 MB, caractere: [PREENCHER]

- 50 MB, palavra: [PREENCHER]
 - 200 MB, caractere: [PREENCHER]
 - 200 MB, palavra: [PREENCHER]
-

14. Conclusão

O programa desenvolvido implementa o algoritmo de Huffman para compressão e descompressão de arquivos-texto em dois modos: por caractere e por palavra. A compactação por caractere trabalha com cada caractere como símbolo, enquanto a compactação por palavra utiliza tokens formados por palavras e separadores.

O arquivo compactado armazena o dicionário Huffman no próprio arquivo, o que permite que a descompactação seja feita corretamente sem depender de informações externas. A implementação também apresenta métricas de tamanho e tempo de execução, permitindo a análise empírica exigida pelo trabalho.

Conclusão final após os testes: [PREENCHER]

15. Referência

Enunciado do Trabalho I de Projeto e Análise de Algoritmos, Ciência da Computação, Campus Foz do Iguaçu, Jun/2026.